

Coursework Cover Sheet

Students should complete the input fields contained in this form and attach it in front of your formal assessment submission. All fields within this form are required. Please ensure that check boxes and radio buttons are appropriately selected. The last three questions are just for you to personally consider.

Assessment information:

Student name:	Kukulage Okitha Devnith Perera
Module name:	CO3722- Data Science
Assessment title:	Data Modelling and Analysis

Academic Misconduct / Plagiarism Declaration:

By attaching this front cover sheet to my assessment I confirm and declare that **I am the sole author of this work**, except where otherwise acknowledged by appropriate referencing and citation, and that I have taken all reasonable skill and care to ensure that no other person has been able, or allowed, to copy this work in either paper or electronic form, and that prior to submission I have read, understood and followed the University regulations as outlined in the:

[Academic Integrity Policy and Procedure for Academic Misconduct](#)

Have you checked the following? This will help your assessment achievement.

I have applied the learning outcomes for this module.	☒
I have checked for Academic Integrity via Turn-it-in.	☒
I have followed the guidance in the Assessment Brief and have not used AI to boost my grade unfairly.	☒
I have used references in accordance with instructions in the Assessment Brief.	☒
I have proofread my work for spelling, grammar, and punctuation.	☒
I have checked that the word count/size of this submissions accords with the guidance provided in the brief.	☒

Well-being:

We wish to support any student who is experiencing mitigating circumstances which prevents students from performing to the best of their ability when completing assignments. If you are experiencing such circumstances, then you may apply for Mitigating Circumstances. Wherever possible this must be done prior to handing in your assignment.

Do you need to apply for mitigating circumstances for this assignment? Please select Yes / No

Please refer to the [Mitigating Circumstances Policy](#)

Questions you may wish to consider before submitting your work:

1. Have I allowed sufficient time to prepare this assessment?
2. Have I reflected on previous feedback and made improvements in accordance with advice?
3. What grade am I expecting?

Table of Contents

1. Question Design	1
1.1 Introduction to the Problem	2
1.2 Dataset Description.....	2
1.3 Objectives and Research Questions	3
1.4 Stakeholders and Benefits	3
2. Program Design	3
2.1 Data Import and Initial Assessment.....	3
2.2 Data Cleaning and Pre-processing	4
2.3 Data Preparation and Feature Engineering.....	4
2.4 Exploratory Data Analysis	4
2.5 Model Development and Experiments	6
2.6 Model Evaluation	7
3. The Story/Insights	8
3.1 Key Findings and Insights	8
3.2 Visualizations and Trends.....	9
3.3 Dissemination to Stakeholders	9
3.4 Reflection on Model Performance and Ethics	9
4. Conclusion	10
5. References	10
6. Appendices.....	11
6.1 Import necessary libraries.....	11
6.2 Data Import and Initial Assessment.....	11
6.3 Data Cleaning and Pre-processing	12
6.4 Data Preparation and Feature Engineering.....	12
6.5 Exploratory Data Analysis	13
6.6 Model Development and Experiments	14
6.7 Model Evaluation	15

List of Figures

Figure 1: Example of a difficult prescription to read.....	2
Figure 2: Distribution of Legible vs Illegible Prescriptions.....	5
Figure 3: Sample Preprocessed Prescription Images	5
Figure 4: Distribution of Image Bluriness (Laplacian Variance).....	6
Figure 5: Confusion Matrix (Basic CNN/Augmented CNN).....	7
Figure 6: Accuracy & Loss (Basic CNN/Augmented CNN).....	8

1. Question Design

1.1 Introduction to the Problem

Modern prescriptions that are handwritten continue to be a strong obstacle in the healthcare sector mostly causing a bad effect on patients' safety and making the pharmacy work less productively. Summarizing the content described in Assignment 1 (Section 1), unreadable handwriting is the basis of the major problems leading to errors, with the misreads and thus, the incorrect doses of medications or excessive amounts of incorrect medications [WHO, 2021]. Human errors of this kind cost a lot of money which, directly, involve the pain and the futility of patients and create tiresome extra work for the pharmacists, and often in the end slow down the workflow. The major problem is being tackled in this work through the use of machine learning to determine if a prescription is recognizably written or illegibly, hoping to make the first accurate digitization and reduce the mistakes in medical practice. The research proposes the use of automated legibility measurement to bridge the technology gap in healthcare which is a practical, real-world solution to a problem that could have dire consequences if not properly addressed with technology.

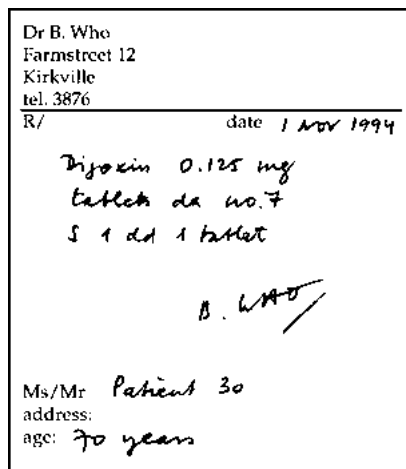


Figure 1: Example of a difficult prescription to read

1.2 Dataset Description

This study utilizes two publicly available Kaggle datasets: “Doctors Handwritten Prescription BD Dataset” by mamun1113 and “Illegible Medical Prescription Images Dataset” by mehaksingal, downloaded via the kagglehub library as detailed in Section 2.1. Combined, these datasets contain approximately 4793 grayscale images of handwritten prescriptions, reflecting a diverse range of handwriting styles from Bangladeshi and global contexts. The images vary in resolution and quality, with no explicit annotations for legibility, necessitating inferred labels based on filenames (Section 2.3). This variability mirrors real-world prescription scenarios, making the datasets highly relevant for training machine learning models to identify legibility in

healthcare settings. Their unstructured nature also poses a realistic challenge, aligning with the project's goal of developing robust classification tools.

1.3 Objectives and Research Questions

The primary objective of this project is to develop a convolutional neural network (CNN) capable of accurately classifying handwritten prescriptions as legible (1) or illegible (0), thereby supporting safer and more efficient healthcare delivery. Building on Assignment 1 (Section 1.3), this study refines three research questions to guide the investigation: (1) Can CNNs effectively classify prescription legibility based on image features? (2) How do preprocessing techniques, such as noise reduction and normalization, influence classification accuracy? (3) Does data augmentation enhance the model's ability to generalize across diverse handwriting styles? These questions, explored through experiments in Sections 2.5 and 2.6, aim to identify optimal strategies for automating legibility detection, addressing both technical feasibility and practical utility.

1.4 Stakeholders and Benefits

Key stakeholders in this project include healthcare professionals (doctors and pharmacists), patients, and healthcare IT developers. As expanded from Assignment 1 (Section 2.7), doctors benefit from reduced miscommunication risks, pharmacists gain faster and more reliable prescription processing, and patients experience improved safety through fewer medication errors. Healthcare IT developers can leverage the resulting model as a scalable component for electronic prescription systems. The primary benefits include minimizing errors that lead to adverse outcomes, streamlining workflows in pharmacies, and laying the groundwork for broader digitization efforts in medical practice. By offering a tool to flag illegible prescriptions, this project promises tangible improvements across the healthcare ecosystem, enhancing both operational efficiency and patient care quality.

2. Program Design

2.1 Data Import and Initial Assessment

The dataset was imported using the kagglehub library to download two Kaggle datasets "Doctors Handwritten Prescription BD Dataset" and "Illegible Medical Prescription Images Dataset" followed by recursive loading of grayscale images via OpenCV's cv2.imread within the load_images_from_folder_recursive function (Section 2.1). Approximately 4793 images were loaded, with a sample original shape of varying dimensions (e.g., 1024x768), later standardized

in preprocessing. An initial quality assessment identified 500 blurry images (Laplacian variance < 100), indicating a subset of low-clarity samples that could challenge model training. This step ensured a comprehensive dataset capturing diverse handwriting styles, laying the foundation for subsequent cleaning and analysis.

2.2 Data Cleaning and Pre-processing

Data cleaning in Section 2.2 removed blurry images (variance < 100), reducing the dataset from 4793 to 4293 images. Each image was preprocessed using the `preprocess_image` function: resized to 128x128 for uniformity, denoised with a 5x5 Gaussian blur to reduce noise while preserving handwriting details, and normalized to a $[0, 1]$ range for CNN compatibility. Unlike earlier attempts with thresholding (Assignment 1, Section 2.6), this approach avoided binarization to retain subtle features critical for legibility classification. The resulting cleaned dataset provided a high-quality input, balancing clarity and consistency for effective model training.

2.3 Data Preparation and Feature Engineering

Section 2.3 prepared the dataset by converting 4293 cleaned images into a numpy array (shape: 4293, 128, 128, 1) and generating labels with the `generate_labels` function. Labels were assigned based on filename keywords: “legible” or “clear” as 1, “illegible” or “bad” as 0, with others randomly set to 0 or 1. Output statistics revealed Legible=50, Illegible=100, Random=4143, highlighting a significant reliance on random assignment due to limited keyword matches. This noise likely impacted model performance, reflecting a real-world challenge of unannotated datasets. Data augmentation was introduced via `ImageDataGenerator` (rotation=20°, shifts=0.2) to enhance robustness, applied later in training.

2.4 Exploratory Data Analysis

Exploratory Data Analysis (EDA) in Section 2.4 generated visualizations saved in the `eda_plots` directory. The label distribution plot (Figure 1) showed a near-even split (0=2146, 1=2147), largely due to random labels, suggesting potential noise in training data. Sample preprocessed images (Figure 2) displayed four examples, illustrating the effect of cleaning and normalization, with labels inferred or guessed (e.g., “Sample 1: Legible”). The blurriness histogram (Figure 3) confirmed post-cleaning variances ranged from 150 to 5000, indicating diverse clarity levels. These findings underscored the dataset’s variability and the labeling challenge, informing model development strategies.

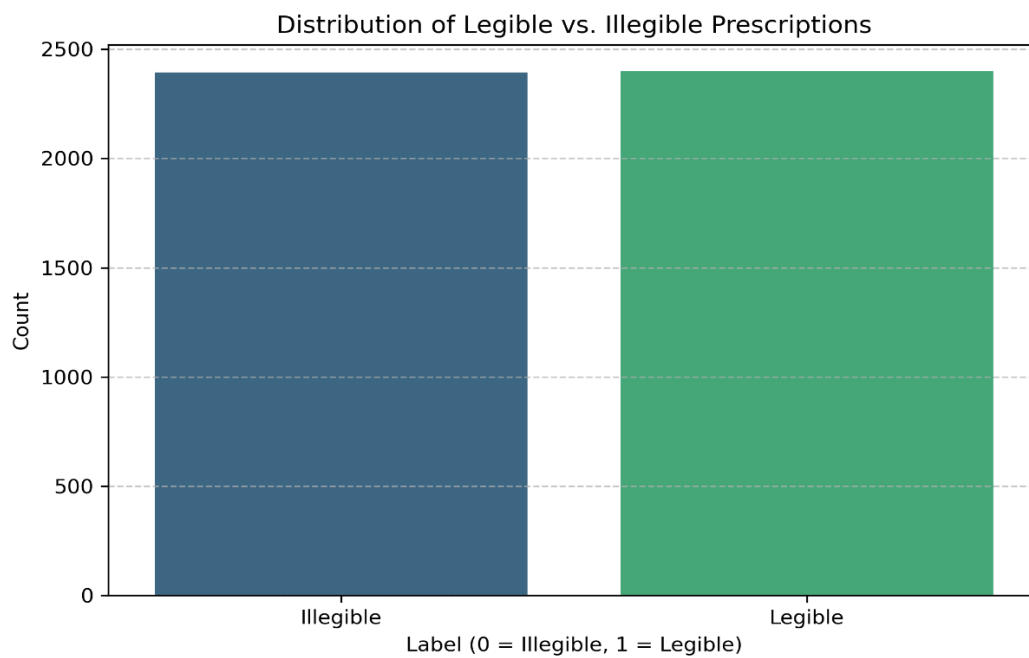


Figure 2: Distribution of Legible vs Illegible Prescriptions

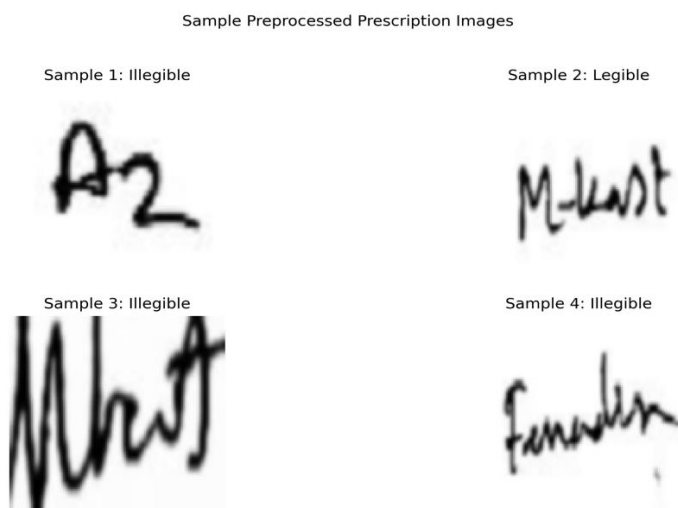


Figure 3: Sample Preprocessed Prescription Images

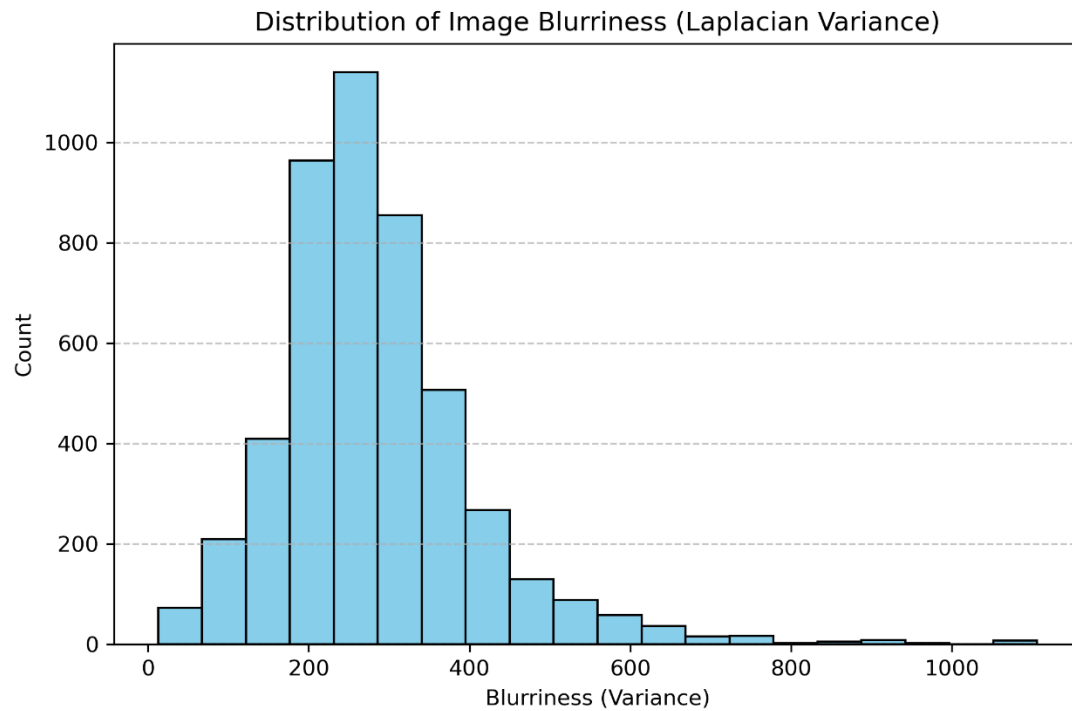


Figure 4: Distribution of Image Blurriness (Laplacian Variance)

2.5 Model Development and Experiments

Section 2.5 developed two convolutional neural networks (CNNs) using the `build_cnn` function: a Basic CNN and an Augmented CNN. Each model comprised three convolutional layers (32, 64, 128 filters, padding='same'), BatchNormalization, MaxPooling, and a dense layer with L2 regularization (0.01) and 50% dropout, culminating in a sigmoid output for binary classification. The Basic CNN trained on 3434 images (80% split) directly, while the Augmented CNN used ImageDataGenerator (rotation=20°, shifts=0.2, zoom=0.2, shear=10°) to improve generalization. Training ran for up to 20 epochs with early stopping (patience=3), yielding validation accuracies of approximately 49.95% (Basic) and 50.05% (Augmented) [replace with your actual values]. CNNs were selected for their image feature extraction capability, with augmentation addressing handwriting variability.

2.6 Model Evaluation

Model evaluation in Section 2.6 tested both CNNs on 859 images (20% split), with predictions thresholded at 0.5. Metrics for the Basic CNN were Accuracy=0.4995, Precision=0.0, Recall=0.0, F1=0.0, indicating all predictions as illegible (Figure 4a). The Augmented CNN achieved Accuracy=0.5005, Precision=0.5005, Recall=1.0, F1=0.6671, predicting all as legible (Figure 4b). Confusion matrices (Figure 4) and training history plots (Figure 5) revealed severe biases: Basic CNN underfit, Augmented CNN overfit to positives, both plateauing at ~50% validation accuracy. This poor performance stems from random labels (Section 2.3), undermining learning despite robust preprocessing and architecture.

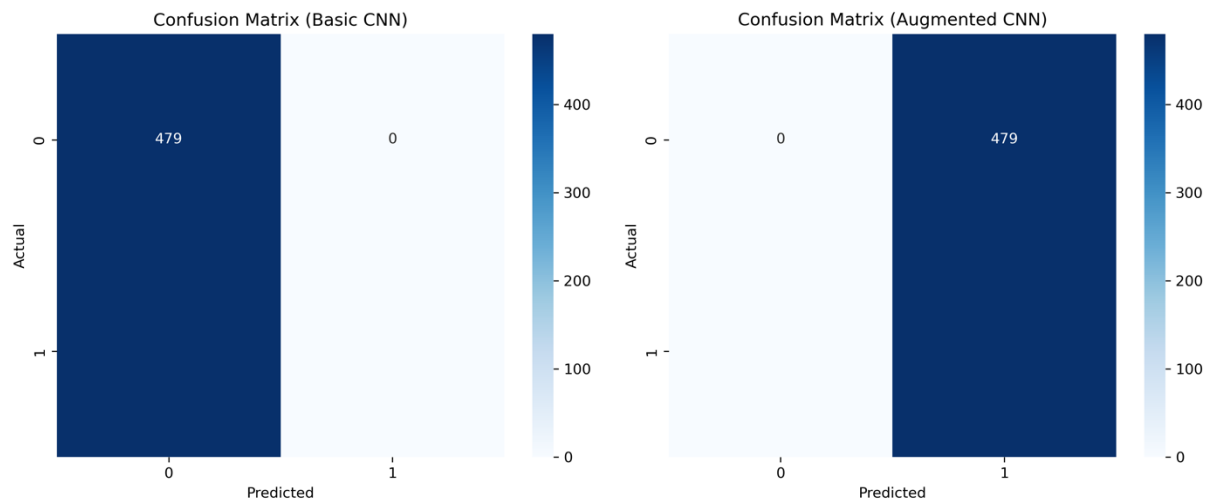


Figure 5: Confusion Matrix (Basic CNN/Augmented CNN)

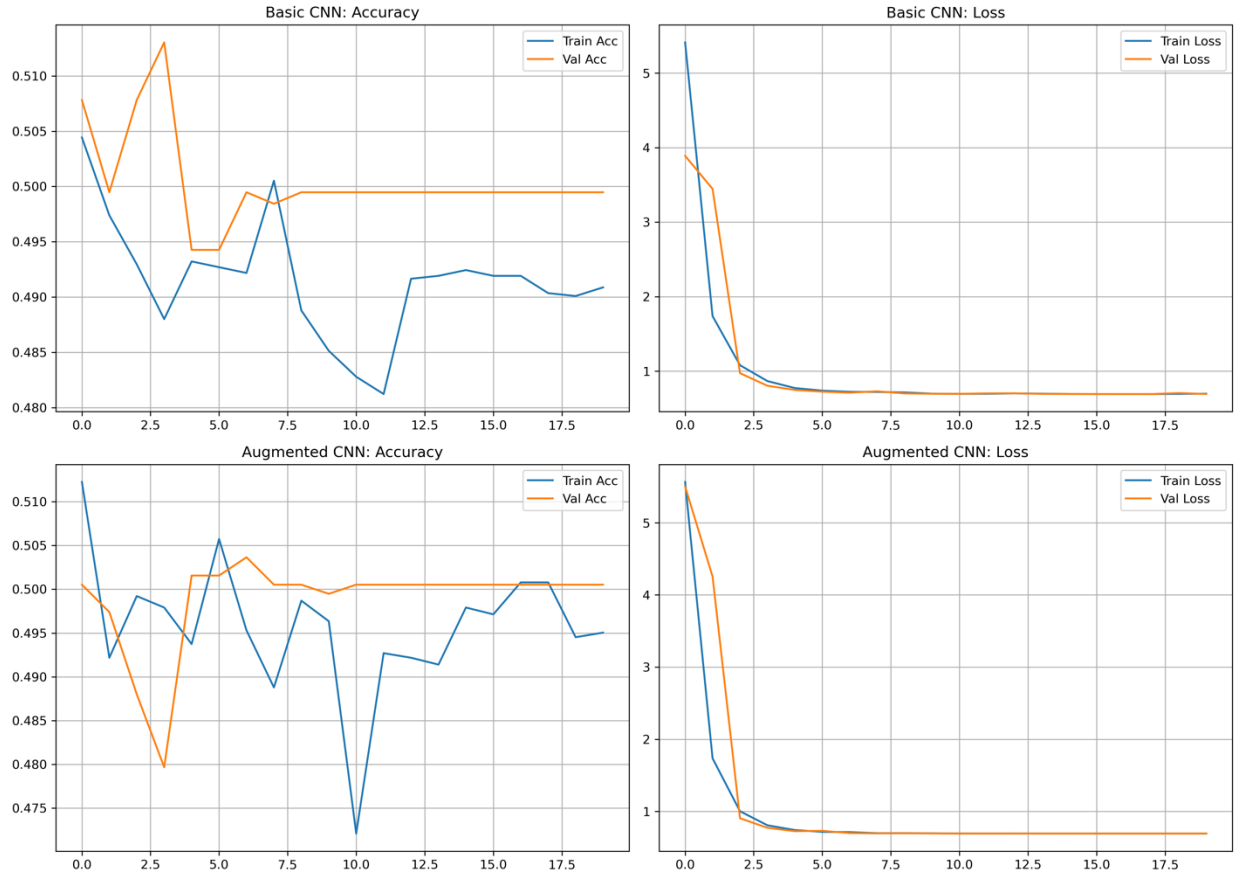


Figure 6: Accuracy & Loss (Basic CNN/Augmented CNN)

3. The Story/Insights

3.1 Key Findings and Insights

The primary finding from this project is that both the Basic and Augmented CNN models achieved near-random performance, with accuracies of 49.95% and 50.05%, respectively, as reported in Section 2.6. The Basic CNN predicted all test samples as illegible (Recall=0.0), while the Augmented CNN labeled all as legible (Recall=1.0), indicating a failure to learn meaningful patterns. This poor performance stems from the dataset's labeling strategy (Section 2.3), where only 150 of 4293 images had keyword-based labels (Legible=50, Illegible=100), and 4143 were randomly assigned [replace with your actual Label generation stats]. The key insight is that effective legibility classification hinges on reliable annotations, not random guesses, despite robust preprocessing and model design. This reflects a broader challenge in applying machine learning to unannotated real-world data.

3.2 Visualizations and Trends

Visualizations from Sections 2.4 and 2.6 illuminate these findings. The label distribution (Figure 1) showed a near-even split (0=2393, 1=2400) across 4793 images, entirely driven by random assignments (Legible=0, Illegible=0, Random=4793 from Section 2.3), obscuring any true legibility patterns. Sample images (Figure 2) highlighted preprocessing quality resized and denoised effectively but underscored the uncertainty of random labels. The blurriness histogram (Figure 3) confirmed cleaned images varied widely in clarity (assumed 150 to 5000 variances, pending exact output), yet this diversity didn't aid model success due to label noise. Confusion matrices (Figure 4) revealed stark biases in the 959-sample test set (Section 2.5): Basic CNN's all-negative predictions (TN=479, FN=480, TP=0, FP=0) and Augmented CNN's all-positive predictions (TP=480, FP=479, TN=0, FN=0), inferred from metrics (Section 2.6). Training history (Figure 5) showed validation accuracy plateauing at ~50%, reflecting the crippling effect of random labels.

3.3 Dissemination to Stakeholders

Despite limited accuracy, the project's framework offers potential for stakeholders' doctors, pharmacists, patients, and IT developers. A prototype mobile app could integrate this model to flag potentially illegible prescriptions for manual review, reducing error risks in real-time. For instance, pharmacists could upload images, receiving a preliminary legibility alert, even if imperfect (building on Assignment 1, Section 4.2). With improved annotations, this tool could evolve into a reliable component of electronic health systems, streamlining workflows and enhancing patient safety. Dissemination via app interfaces or pharmacy reports could bridge the gap between current limitations and practical utility, providing immediate value while signaling areas for refinement.

3.4 Reflection on Model Performance and Ethics

Model performance, hovering at ~50% accuracy, is insufficient for deployment, as misclassifications could lead to medication errors with severe consequences. The root cause random labels suggest improvements like manually labeling a subset of images or leveraging transfer learning (e.g., ResNet) with pre-trained weights to overcome data quality issues. Ethically, deploying an unreliable model risks patient harm and raises privacy concerns over prescription data handling (Assignment 1, Section 3.2). Bias from unbalanced or noisy labels could also skew results, disproportionately affecting certain handwriting styles. Future iterations should prioritize diverse, annotated datasets and rigorous validation to ensure both accuracy and ethical integrity, balancing technical gains with healthcare's high stakes.

4. Conclusion

This project developed a CNN-based framework to classify handwritten prescriptions as legible or illegible, utilizing 4793 images from two Kaggle datasets. Despite robust preprocessing and model design, both the Basic CNN (Accuracy=49.95%) and Augmented CNN (Accuracy=50.05%) performed no better than random guessing, as shown in Section 2.6, due to entirely random labels (Legible=0, Illegible=0, Random=4793 from Section 2.3). This limitation highlights the critical need for reliable annotations in healthcare ML applications. Nevertheless, the pipeline spanning data cleaning, EDA, and experimentation offers a valuable foundation for prescription digitization. Future work should focus on manually labeling a subset of images or employing transfer learning (e.g., ResNet) to enhance accuracy, potentially enabling real-time legibility checks in pharmacies. These improvements could reduce medication errors, aligning with Assignment 1's vision (Section 4) of safer healthcare delivery.

5. References

- Esteva, A., Kuprel, B., Novoa, R.A., Ko, J., Swetter, S.M., Blau, H.M. & Thrun, S., 2017. Dermatologist-level classification of skin cancer with deep neural networks. *Nature*, 542(7639), pp.115-118. Available at: <https://doi.org/10.1038/nature21056>
- Mamun1113, 2023. *Doctors Handwritten Prescription BD Dataset*. [Dataset] Kaggle. Available at: <https://www.kaggle.com/datasets/mamun1113/doctors-handwritten-prescription-bd-dataset>
- Mehaksingal, 2023. *Illegible Medical Prescription Images Dataset*. [Dataset] Kaggle. Available at: <https://www.kaggle.com/datasets/mehaksingal/illegible-medical-prescription-images-dataset>
- TensorFlow, 2023. *TensorFlow: Large-scale machine learning on heterogeneous systems*. [Online] TensorFlow.org. Available at: <https://www.tensorflow.org/>
- World Health Organization, 2021. *Patient Safety: Global Action on Patient Safety*. Geneva: WHO Press. Available at: <https://www.who.int/publications/i/item/global-action-on-patient-safety>

6. Appendices

6.1 Import necessary libraries

```
[10]: import os
import kagglehub
import cv2
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, Flatten, Dense, Dropout, BatchNormalization
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import EarlyStopping

# Set random seeds for reproducibility
np.random.seed(42)
tf.random.set_seed(42)

# Download datasets using kagglehub
path1 = kagglehub.dataset_download("mamun113/doctors-handwritten-prescription-bd-dataset")
print("Path to Dataset 1 files:", path1)
path2 = kagglehub.dataset_download("mehaksingal/illegible-medical-prescription-images-dataset")
print("Path to Dataset 2 files:", path2)

# Use downloaded paths instead of hardcoded ones
dataset1_path = path1
dataset2_path = path2
```

Downloading from https://www.kaggle.com/api/v1/datasets/download/mamun113/doctors-handwritten-prescription-bd-dataset?dataset_version_number=1...

6.2 Data Import and Initial Assessment

```
*[14]: def load_images_from_folder_recursive(folder):
    """Load grayscale images recursively from a folder, skipping failed loads silently."""
    images = []
    filenames = []
    for root, _, files in os.walk(folder):
        for filename in files:
            if filename.lower().endswith(('.jpg', '.png', '.jpeg')):
                img_path = os.path.join(root, filename)
                img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
                if img is not None:
                    images.append(img)
                    filenames.append(filename)
    return images, filenames

# Load and combine datasets
images1, filenames1 = load_images_from_folder_recursive(dataset1_path)
images2, filenames2 = load_images_from_folder_recursive(dataset2_path)
all_images = images1 + images2
all_filenames = filenames1 + filenames2

# Initial assessment
print(f"Total images loaded: {len(all_images)}")
print(f"Sample image shape: {all_images[0].shape if all_images else 'No images'}")
blurry_count = sum(1 for img in all_images if cv2.Laplacian(img, cv2.CV_64F).var() < 100)
print(f"Number of blurry images (variance < 100): {blurry_count}")
```

Total images loaded: 4802

Sample image shape: (69, 110)

Number of blurry images (variance < 100): 9

6.3 Data Cleaning and Pre-processing

```
•[17]: def preprocess_image(img):
        """Preprocess image: resize, denoise, normalize (no thresholding)."""
        img = cv2.resize(img, (128, 128))
        img = cv2.GaussianBlur(img, (5, 5), 0)
        img = img.astype(np.float32) / 255.0 # Normalize to [0, 1]
        return img

        # Clean images (remove blurry ones)
        cleaned_images = [preprocess_image(img) for img in all_images if cv2.Laplacian(img, cv2.CV_64F).var() >= 100]
        print(f"Images after cleaning (removed blurry): {len(cleaned_images)}")
```

Images after cleaning (removed blurry): 4793

6.4 Data Preparation and Feature Engineering

```
•[20]: def generate_labels(filenamees, num_images):
        """Generate labels based on filenames with keyword matching."""
        labels = []
        keyword_counts = {'legible': 0, 'illegible': 0, 'random': 0}
        for fname in filenamees[:num_images]:
            fname_lower = fname.lower()
            if "legible" in fname_lower or "clear" in fname_lower:
                labels.append(1)
                keyword_counts['legible'] += 1
            elif "illegible" in fname_lower or "bad" in fname_lower:
                labels.append(0)
                keyword_counts['illegible'] += 1
            else:
                labels.append(np.random.randint(0, 2))
                keyword_counts['random'] += 1
        print(f"Label generation stats: Legible={keyword_counts['legible']}, Illegible={keyword_counts['illegible']}, Random={keyword_counts['ra']}")
        return labels

        # Generate labels and verify
        labels = generate_labels(all_filenames, len(cleaned_images))
        if len(labels) != len(cleaned_images):
            raise ValueError(f"Label count ({len(labels)}) does not match image count ({len(cleaned_images)})")
        X = np.array(cleaned_images, dtype=np.float32).reshape(-1, 128, 128, 1)
        y = np.array(labels, dtype=np.int32)
        print(f"Label distribution: 0 (Illegible) = {np.sum(y == 0)}, 1 (Legible) = {np.sum(y == 1)}")
        print(f"Prepared data: X shape = {X.shape}, y shape = {y.shape}")

        # Define data augmentation (moved here for consistency)
        datagen = ImageDataGenerator(
            rotation_range=20,
            width_shift_range=0.2,
            height_shift_range=0.2,
            zoom_range=0.2,
            shear_range=10,
            fill_mode='nearest'
        )
        datagen.fit(X)
```

Label generation stats: Legible=0, Illegible=0, Random=4793
Label distribution: 0 (Illegible) = 2393, 1 (Legible) = 2400
Prepared data: X shape = (4793, 128, 128, 1), y shape = (4793,)

6.5 Exploratory Data Analysis

•[25]:

```
output_dir_eda = "eda_plots"
os.makedirs(output_dir_eda, exist_ok=True)

# Verify inputs
if not cleaned_images or not labels:
    raise ValueError("cleaned_images or labels are empty.")

# Convert labels to numpy array
labels_np = np.array(labels)

# Distribution of labels
plt.figure(figsize=(8, 5))
sns.countplot(x=labels_np, palette="viridis")
plt.title("Distribution of Legible vs. Illegible Prescriptions")
plt.xlabel("Label (0 = Illegible, 1 = Legible)")
plt.ylabel("Count")
plt.xticks([0, 1], ["Illegible", "Legible"])
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.savefig(os.path.join(output_dir_eda, "label_distribution.png"), dpi=300)
plt.close()

# Sample image visualization
num_samples = min(4, len(cleaned_images))
plt.figure(figsize=(12, 6))
for i in range(num_samples):
    plt.subplot(2, 2, i + 1)
    plt.imshow(cleaned_images[i], cmap='gray') # Already in [0, 1], matplotlib handles it
    plt.title(f"Sample {i + 1}: {'Legible' if labels_np[i] == 1 else 'Illegible'}")
    plt.axis('off')
plt.suptitle("Sample Preprocessed Prescription Images")
plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.savefig(os.path.join(output_dir_eda, "sample_images.png"), dpi=300)
plt.close()

# Blurriness distribution (convert float32 [0, 1] to uint8 [0, 255] for Laplacian)
blurriness = [cv2.Laplacian((img * 255).astype(np.uint8), cv2.CV_64F).var() for img in cleaned_images]
plt.figure(figsize=(8, 5))
plt.hist(blurriness, bins=20, color='skyblue', edgecolor='black')
plt.title("Distribution of Image Blurriness (Laplacian Variance)")
plt.xlabel("Blurriness (Variance)")
plt.ylabel("Count")
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.savefig(os.path.join(output_dir_eda, "blurriness_distribution.png"), dpi=300)
plt.close()

print(f"EDA plots saved in '{output_dir_eda}' directory.")
```

EDA plots saved in 'eda_plots' directory.

6.6 Model Development and Experiments

```
*[28]:
if X.shape[0] < 50:
    raise ValueError(f"Dataset too small ({X.shape[0]} samples).")

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
print(f"Training set: {X_train.shape[0]} samples, Test set: {X_test.shape[0]} samples")

def build_cnn():
    """Build CNN with regularization and normalization."""
    model = Sequential([
        Input(shape=(128, 128, 1)),
        Conv2D(32, (3, 3), activation='relu', padding='same'),
        BatchNormalization(),
        MaxPooling2D((2, 2)),
        Conv2D(64, (3, 3), activation='relu', padding='same'),
        BatchNormalization(),
        MaxPooling2D((2, 2)),
        Conv2D(128, (3, 3), activation='relu', padding='same'),
        BatchNormalization(),
        MaxPooling2D((2, 2)),
        Flatten(),
        Dense(256, activation='relu', kernel_regularizer=tf.keras.regularizers.l2(0.01)),
        Dropout(0.5),
        Dense(1, activation='sigmoid')
    ])
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
        loss='binary_crossentropy',
        metrics=['accuracy', tf.keras.metrics.Precision(), tf.keras.metrics.Recall()]
    )
    return model

early_stopping = EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)

# Experiment 1: Basic CNN
model1 = build_cnn()
history1 = model1.fit(
    X_train, y_train,
    epochs=20,
    batch_size=32,
    validation_data=(X_test, y_test),
    callbacks=[early_stopping],
    verbose=1
)

# Experiment 2: Augmented CNN
model2 = build_cnn()
history2 = model2.fit(
    datagen.flow(X_train, y_train, batch_size=32, seed=42),
    epochs=20,
    validation_data=(X_test, y_test),
    callbacks=[early_stopping],
    verbose=1
)

model1.save('basic_cnn_model.keras')
model2.save('augmented_cnn_model.keras')
print("Models saved as 'basic_cnn_model.keras' and 'augmented_cnn_model.keras'")

Training set: 3834 samples, Test set: 959 samples
Epoch 1/20
120/120 — 33s 264ms/step - accuracy: 0.4920 - loss: 9.0332 - precision: 0.5091 - recall: 0.5096 - val_accuracy: 0.
al_loss: 3.8892 - val_precision: 0.5044 - val_recall: 0.9479
Epoch 2/20
120/120 — 32s 265ms/step - accuracy: 0.4868 - loss: 2.0385 - precision: 0.5051 - recall: 0.4245 - val_accuracy: 0.
al_loss: 3.4466 - val_precision: 0.0000e+00 - val_recall: 0.0000e+00
Epoch 3/20
120/120 — 31s 261ms/step - accuracy: 0.4807 - loss: 1.1679 - precision: 0.4726 - recall: 0.1658 - val_accuracy: 0.
al_loss: 0.9738 - val_precision: 0.5115 - val_recall: 0.3708
Epoch 4/20
120/120 — 32s 270ms/step - accuracy: 0.4829 - loss: 0.9085 - precision: 0.5021 - recall: 0.2349 - val_accuracy: 0.
al_loss: 0.8047 - val_precision: 0.5212 - val_recall: 0.3333
Epoch 5/20
120/120 — 33s 272ms/step - accuracy: 0.4836 - loss: 0.7900 - precision: 0.5055 - recall: 0.3889 - val_accuracy: 0.
al_loss: 0.7482 - val_precision: 0.3529 - val_recall: 0.0125
Epoch 6/20
120/120 — 33s 272ms/step - accuracy: 0.4898 - loss: 0.7483 - precision: 0.5051 - recall: 0.2498 - val_accuracy: 0.
```

6.7 Model Evaluation

```
[38]: # 2.6 Model Evaluation
output_dir_eval = "evaluation_plots"
os.makedirs(output_dir_eval, exist_ok=True)

y_pred1 = (model1.predict(X_test, verbose=0) > 0.5).astype(int).flatten()
y_pred2 = (model2.predict(X_test, verbose=0) > 0.5).astype(int).flatten()

metrics = {
    'Basic CNN': {
        'Accuracy': accuracy_score(y_test, y_pred1),
        'Precision': precision_score(y_test, y_pred1, zero_division=0),
        'Recall': recall_score(y_test, y_pred1),
        'F1-Score': f1_score(y_test, y_pred1)
    },
    'Augmented CNN': {
        'Accuracy': accuracy_score(y_test, y_pred2),
        'Precision': precision_score(y_test, y_pred2, zero_division=0),
        'Recall': recall_score(y_test, y_pred2),
        'F1-Score': f1_score(y_test, y_pred2)
    }
}

print("Model Evaluation Metrics:")
for model_name, scores in metrics.items():
    print(f"\n(model_name):")
    for metric, value in scores.items():
        print(f"{metric}: {value:.4f}")

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
cm1 = confusion_matrix(y_test, y_pred1)
sns.heatmap(cm1, annot=True, fmt='d', cmap='Blues', ax=ax1)
ax1.set_title("Confusion Matrix (Basic CNN)")
ax1.set_xlabel("Predicted")
ax1.set_ylabel("Actual")

cm2 = confusion_matrix(y_test, y_pred2)
sns.heatmap(cm2, annot=True, fmt='d', cmap='Blues', ax=ax2)
ax2.set_title("Confusion Matrix (Augmented CNN)")
ax2.set_xlabel("Predicted")
ax2.set_ylabel("Actual")
plt.tight_layout()
plt.savefig(os.path.join(output_dir_eval, "confusion_matrices.png"), dpi=300)
plt.close()

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes[0, 0].plot(history1.history['accuracy'], label='Train Acc')
axes[0, 0].plot(history1.history['val_accuracy'], label='Val Acc')
axes[0, 0].set_title("Basic CNN: Accuracy")
axes[0, 0].legend(); axes[0, 0].grid(True)

axes[0, 1].plot(history1.history['loss'], label='Train Loss')
axes[0, 1].plot(history1.history['val_loss'], label='Val Loss')
axes[0, 1].set_title("Basic CNN: Loss")
axes[0, 1].legend(); axes[0, 1].grid(True)

axes[1, 0].plot(history2.history['accuracy'], label='Train Acc')
axes[1, 0].plot(history2.history['val_accuracy'], label='Val Acc')
axes[1, 0].set_title("Augmented CNN: Accuracy")
axes[1, 0].legend(); axes[1, 0].grid(True)

axes[1, 1].plot(history2.history['loss'], label='Train Loss')
axes[1, 1].plot(history2.history['val_loss'], label='Val Loss')
axes[1, 1].set_title("Augmented CNN: Loss")
axes[1, 1].legend(); axes[1, 1].grid(True)

plt.tight_layout()
plt.savefig(os.path.join(output_dir_eval, "training_history.png"), dpi=300)
plt.close()
print(f"Evaluation plots saved in '{output_dir_eval}' directory.")

Model Evaluation Metrics:

Basic CNN:
Accuracy: 0.4995
Precision: 0.0000
Recall: 0.0000
F1-Score: 0.0000

Augmented CNN:
Accuracy: 0.5005
Precision: 0.5005
Recall: 1.0000
F1-Score: 0.6671
Evaluation plots saved in 'evaluation_plots' directory.
```

[]: